

5. 코드 안내 — 받으신 코드 zip 따라가기

이 자료와 함께 전달된 **코드 zip** (`study/wildfire-poc/`) 의 구조와 읽는 순서.

1. 전체 폴더 트리

study/wildfire-poc/	
├─ README.md	← 환경 셋업
├─ run.sh	← Python 스크립트 wrapper
├─ db/	← 🗄️ DB 계층
│ ├─ schema_v2.tql	← 5종 엔티티 정의 (현행)
│ ├─ functions.tql	← fun (5개) - 자족 추론
│ ├─ apply_schema_v2.py	← 스키마 적용
│ ├─ apply_functions.py	← 함수 적용
│ ├─ seed_params.py	← threshold v1/v2 + weight 적재
│ ├─ load_sources.py	← source 29건 적재
│ ├─ load_segments.py	← segment 5건 적재
│ └─ check.py / check_all.py	← 조회 도구
├─ inference/	← 🏳️ 추론 계층
│ ├─ formulas.py	← 점수 산식 (Python)
│ ├─ graph_inference.py	← Python 추론 (As-is)
│ ├─ graph_inference_fun.py	← fun 추론 (To-be) ★
│ └─ run_inference_demo.py	← 추론 실행 + write-back
├─ llm/	← 💬 LLM 계층
│ ├─ text_to_typeql.py	← 자연어 → TypeQL (Gemini)
│ ├─ chat.py	← 대화 + advisory
│ └─ eval/run_eval.py	← 평가셋 14건 자동 실행
├─ demo/	← 🌐 데모 웹 (FastAPI, :8000)
│ ├─ server.py	
│ ├─ api/demo1.py ~ demo4.py	← 시연 ①~④ API
│ └─ run_demo.sh	
├─ tests/	← ✅ 회귀 테스트
│ ├─ test_python_vs_fun.py	← Python ↔ fun 일치 검증
│ ├─ test_param_externalization.py	← v1↔v2 정책 전환 검증
│ └─ test_golden_graph.py	
├─ _spike/	← 🖋️ 실험 (TypeQL fun 검증)
│ └─ 01~02d 단계별 spike	
└─ scripts/	← 🛠️ 유틸 (DB 리셋 등)

포트 메모: TypeDB 는 localhost:1729 , 데모 웹은 localhost:8000 — 다른 포트.

2. 계층별 책임

계층	폴더	책임
DB	db/	스키마·함수 정의, 데이터 적재, 조회
추론	inference/	점수 계산, 그래프 write-back
LLM	llm/	자연어 ↔ TypeQL, 평가
데모	demo/	FastAPI 웹 시연
테스트	tests/	회귀 검증
실험	_spike/	TypeDB 3.0 fun 도입 전 검증 (참고용)

3. 환경 셋업

Docker — TypeDB 기동

```
docker run -d --name typedb -p 1729:1729 typedb/typedb:3.4.0
```

Python 환경

```
cd study/wildfire-poc
uv sync # 의존성 설치
```

API 키 설정

```
export GEMINI_API_KEY="your-key-here"
```

초기 데이터 적재 (한 번만)

```
./run.sh db/apply_schema_v2.py
./run.sh db/apply_functions.py
./run.sh db/seed_params.py
./run.sh db/load_sources.py
./run.sh db/load_segments.py
```

4. 코드 읽는 순서 가이드

코드를 처음 접하는 사람에게 권장 순서:

① README.md	환경 셋업
▼	
② db/schema_v2.tql	"어떤 데이터가 있는가" (5종 엔티티)
▼	
③ db/functions.tql	"어떻게 추론하는가" (fun)
▼	
④ db/seed_params.py	"어떤 정책 카드가 있는가" (v1/v2)
▼	
⑤ inference/formulas.py	"점수가 어떻게 계산되는가"
▼	
⑥ inference/graph_inference.py	"Python 추론 어떻게 도는가"
▼	
⑦ inference/run_inference_demo.py	"실제 실행 + write-back"
▼	
⑧ llm/chat.py	"사용자 질의가 어떻게 처리되는가"
▼	
⑨ tests/test_python_vs_fun.py	"Python ↔ fun 결과 일치 검증"

5. 시연 ↔ 코드 매핑

시연	실행 명령	호출되는 파일
시연 1 (write-back)	run.sh db/check.py run.sh inference/run_inference_demo.py	db/check.py , formulas.py , graph_inference.py
시연 2 (fun 듀얼)	run.sh tests/test_param_externalization.py run.sh tests/test_python_vs_fun.py	seed_params.py , graph_inference.py , graph_inference_fun.py
시연 3 (chat + advisory)	run.sh llm/chat.py	chat.py , text_to_typeql.py
시연 4 (평가 가셋 14건)	run.sh llm/eval/run_eval.py	위 모두 + 평가셋

데모 웹으로 보고 싶다면

```
./demo/run_demo.sh
# → http://localhost:8000 접속
```

시연	데모 API
①	demo/api/demo1.py
②	demo/api/demo2.py
③	demo/api/demo3.py
④	demo/api/demo4.py

6. 왜 이 구조인가 — "사실 vs 표현 분리" 의 코드 구현

계층	의미
db/	사실의 보관소 (스키마 + 데이터 + fun)
inference/	사실을 사실로 변환 (점수 계산 + write-back)
llm/	사실을 표현으로 변환 (자연어 답변)
demo/	표현의 시각화 (웹 UI)
tests/	사실 일관성 검증

각 계층은 독립적으로 교체 가능:

- LLM 모델 교체 → llm/text_to_typeql.py 만 수정
 - 추론을 Python → fun 으로 완전 이전 → inference/graph_inference_fun.py 로 스위치
- 이게 본 시스템의 모듈성 설계의 핵심 이다.

7. 환경 트러블슈팅 (자주 나오는 이슈)

증상	원인 / 대처
TYPEDB_UNAVAILABLE 에러	TypeDB 컨테이너 미기동 → <code>docker ps</code> 확인, <code>docker restart typedb</code>
GEMINI_API_KEY 없음	환경변수 미설정 → <code>export GEMINI_API_KEY=...</code>
segment 조회 결과 없음	초기 적재 미실행 → § 3 의 "초기 데이터 적재" 5줄 재실행
포트 1729 충돌	다른 컨테이너 실행 중 → <code>docker stop \$(docker ps -q --filter publish=1729)</code>

 한 줄 요약

"5계층(DB·추론·LLM·데모·테스트) + 1실험(`_spike`). TypeQL 은 `db/` 안에만, 나머지는 Python. `run.sh` 가 모든 입구."